```
{
    /**
     * Update the password for the user.
     *
     * @param  Request  $request
     * @return Response
     */
    public function update(Request $request)
    {
       // Validate the new password length...
        $request->user()->fill([
            'password' => Hash::make($request->newPassword)
        ])->save();    }}
```

If you are using the Bcrypt algorithm, the 'make' method will allow you to manage the 'work factor' of the algorithm using the 'round' option. You can manipulate these variables to hash your password according to your specification:

```
$hashed = Hash::make('password', [
    'rounds' => 12
]);
```

If you use the Argon2 algorithm, you can adjust the 'memory', 'time' and 'threads' options to control the work factor of the algorithm.

```
$hashed = Hash::make('password', [
    'memory' => 1024,
    'time' => 2,
    'threads' => 2,
]);
```

The standard 'check' method can help you verify a hash against a password.

```
if (Hash::check('plain-text', $hashedPassword)) {
   // The passwords match...
}
```

Sometimes, a password will need to be rehashed. This can be done simply by readjusting the work factor in either of the algorithms. The 'needsRehash' function allows you to determine if the work factor has changed since the password was hashed: